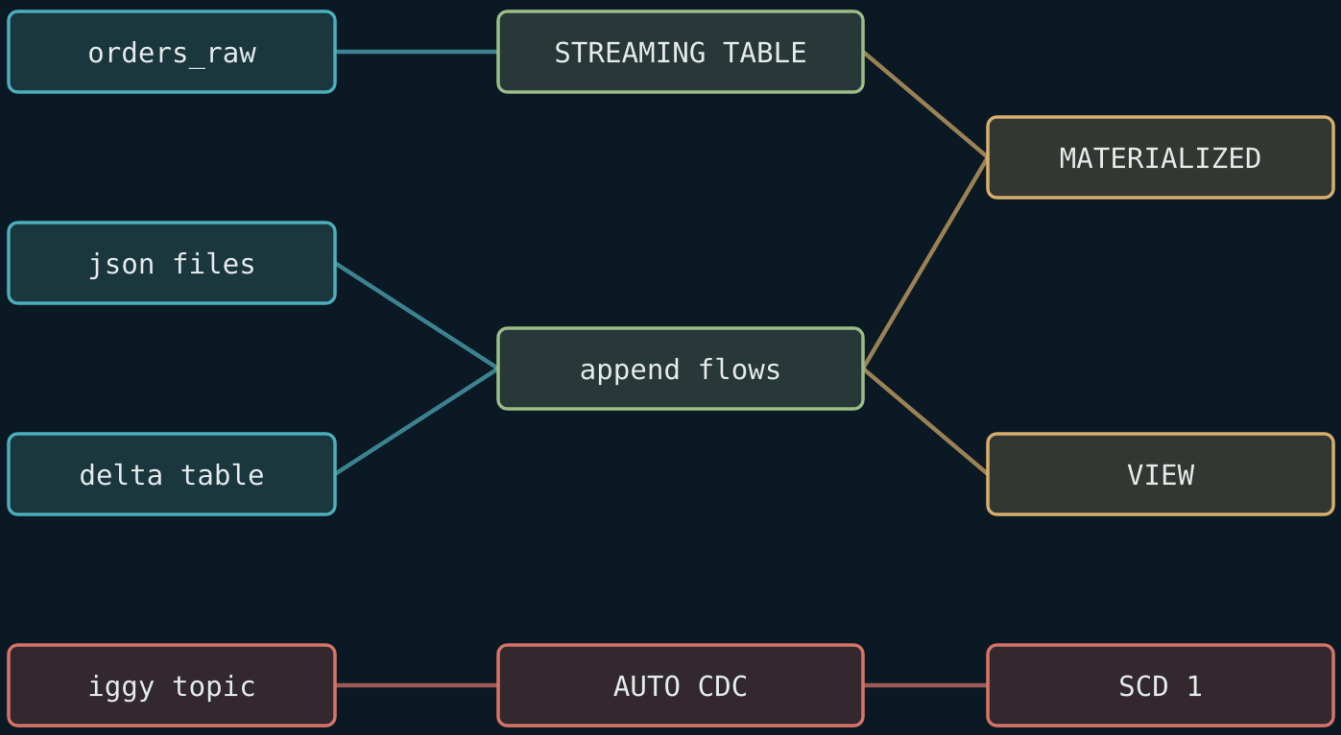


FIRST PAIR PRESS

Sail Declarative Pipelines

Spark 4.2.1 Declarative Pipelines on Sail

Technical Report · Milestone 1

Alexy Khrabrov ∈ LakeSail Team

Sail Declarative Pipelines

Spark 4.2.1 Declarative Pipelines on Sail: Technical Report, Milestone 1

Alexy Khrabrov ∈ LakeSail Team

2026-11-03

Version 1.0.1-bc96ca90
Source commit bc96ca9017ebf7ecddd3cae4404359e6acaf5e1e

First Pair Press
firstpair.press

Contents

1 Summary 3
1.1 What milestone 1 delivers 3
1.2 Headline results 3
2 Background: Spark Declarative Pipelines in 4.2.1 4
2.1 The model 4
2.2 The protocol 5
2.3 What an implementation must provide 5
3 Architecture 6
3.1 Where SDP lives 6
3.2 Crates touched 6
3.3 The `pipelines` module 8
3.4 Run phases 8
4 Components 9
4.1 Graph registration and identifiers 9
4.2 SQL pipeline files 10
4.3 Analysis and validation order 11
4.4 Flow configuration fusion 12
4.5 Materialization and catalog publication 13
4.6 Batch execution 13
4.7 Errors and Spark fidelity 14
4.8 Streaming 15
4.9 AutoCDC 16
4.10 Session lifecycle, shutdown and effect ownership 18
4.11 Cluster mode 19
4.12 ORC 20
4.13 TIME type 20
4.14 Catalogs 21
5 Verification methodology 21
5.1 Principles 21
5.2 Frozen ports 22
5.3 Gate steps 22
5.4 Whole-tree comparison against upstream 22
5.5 Spark oracle runs 23
5.6 Crash seeds and local-cluster 23
5.7 Results on the root 24
6 Differences from Spark 24
6.1 Strict expected failures 24
6.2 Other deliberate differences 25
7 Scope decisions, limits and open work 25
7.1 Scope the user set 25
7.2 Architectural limits 25
7.3 Open work 26
8 The demo 26
8.1 Running it 27
8.2 What it does 27
8.3 What it found 28

1 Summary

Spark Declarative Pipelines (SDP) is the pipeline framework that ships with Apache Spark 4.2.1. A user declares datasets (materialized views, streaming tables, views, sinks) and the flows that fill them, in Python or SQL. Spark builds a dataflow graph from the declarations, resolves and validates it as a whole, then executes it in dependency order. The client is `pyspark.pipelines` together with the `spark-pipelines` command-line tool. Both talk to the server over Spark Connect.

Sail Declarative Pipelines is Sail's implementation of the server side of that protocol. Sail is a Spark Connect server written in Rust on DataFusion. After this milestone, the unchanged `pyspark.pipelines` client and the unchanged `spark-pipelines` CLI can register, analyze, dry-run and run pipelines against Sail. No client patch is needed.

1.1 What milestone 1 delivers

Milestone 1 is the complete batch implementation, plus AutoCDC and streaming.

- **Batch.** Pipelines defined in Python or SQL can be registered, analyzed, dry-run and run. The supported datasets are materialized views, tables, temporary views and persisted views. A SQL file registers all-or-nothing and supports Spark's `USE SCHEMA` and `USE CATALOG` directives. Per-flow SQL settings are fused with the settings of the views a flow reads, as in Spark. Error conditions and the event stream match Spark.
- **Catalogs.** The memory catalog, a Hive 4.0.1 metastore, a second catalog, Unity Catalog and Iceberg are covered. Sessions can share one in-memory catalog through `catalog.shared_memory`, which is off by default.
- **Formats.** Parquet, CSV, JSON, Delta (also as a streaming source) and ORC (read and write).
- **Streaming and CDC.** Bounded microbatches run with the semantics of Spark's AvailableNow trigger. Apache Iggy is a source, through a protocol-neutral `BoundedSource` trait. AutoCDC SCD Type 1 writes into Delta, Iceberg, Hive and Unity targets. It keeps real state between batches for watermarked aggregates, sliding windows, deduplication and inner stream joins.
- **Lifecycle.** Session release and server shutdown never delete what they cannot confirm is finished. Cluster mode is covered, including canceled jobs and bounds on how long stopping a worker can take.

1.2 Headline results

All figures are from `MILESTONE-1.md`.

Measure	Result
Branch	`work/spark-declarative-pipelines`, tag `sdp-batch-m1`, root `fb3a6ab03`
Size	371 commits over upstream `0ba425c03`; 801 files changed
Frozen ports	300 selected public tests and 67 upstream tests: PASS
Pipelines and ORC suite	1262 passed, 23 skipped, 4 strict expected failures
Hive metastore profile	69 passed, 2 xfailed
Second-catalog profile	13 passed
Whole Spark Python tree (CDC gate on the milestone commit `7d6bc1917`)	12134 passed, 99 skipped, 795 xfailed, 0 errors
Local-cluster / Iceberg / Hive / Unity (CDC gate on `7d6bc1917`)	96 / 87 / 89 / 230 passed
Crash seeds	every recovery exact
Spark 4.2.1 oracle, batch	61 files; only the 4 documented batch differences fail on Spark
Spark 4.2.1 oracle, AutoCDC	41 passed; 43 passed and 4 Sail-only skipped
Demo, one machine, with Iggy	48 checks passed, 0 failed, 0 skipped

The four strict expected failures are deliberate differences from Spark. Chapter 6 lists them with their reasons.

2 Background: Spark Declarative Pipelines in 4.2.1

2.1 The model

An SDP pipeline is a graph of datasets and flows.

- A **materialized view** publishes the result of a batch query. A refresh recomputes it.
- A **streaming table** receives incremental writes from one or more flows.
- A **temporary view** is a logical intermediate that is private to the pipeline.
- A **persisted view** is published to the catalog as a view.
- A **sink** writes to an external system.
- A **flow** is a query with a target dataset. A dataset normally has one flow with its own name. A streaming table can have several append flows.

A project spec supplies source globs, a storage URI, the default catalog and database, and SQL configuration. Python and SQL definitions can be mixed. Spark 4.2.1 extends the 4.1 surface with injected `spark` bindings in Python definitions, AutoCDC SCD Type 1, clustering metadata and structured identifiers in the protocol (DESIGN.md).

2.2 The protocol

The client registers the graph through Spark Connect pipeline commands, then starts a run. Spark 4.2.1's `PipelinesHandler` dispatches these commands:

Command	Purpose
`CreateDataflowGraph`	Create a graph with a default catalog, database and SQL config. Returns a graph ID.
`DefineOutput`	Declare a dataset or sink: name, type, schema, format, partitioning, properties.
`DefineFlow`	Declare a flow: name, target, relation (or AutoCDC details) and SQL config.
`DefineSqlGraphElements`	Register the contents of one SQL pipeline file.
`StartRun`	Analyze the graph and, unless it is a dry run, execute it. Streams events back.
`DropDataflowGraph`	Discard a graph.

The proto also declares deferred query-function signal/result commands and `ExecuteOutputFlows`. The 4.2.1 handler does not dispatch them. Sail matches the shipped client and server first and tracks those messages separately (DESIGN.md).

`StartRun` returns a stream. The server sends pipeline events (`PipelineEventResult`, a timestamp and a message) as the run progresses, then a final command result.

2.3 What an implementation must provide

DESIGN.md states the semantic contract. In short:

1. **Declarative registration.** Registration records definitions. It never executes DDL. Identifiers are parsed into parts and never split on quoted dots.
2. **Isolated analysis.** Graph-local names bind before catalog names. Temporary views stay private. All flows are analyzed before any write. Per-flow configuration never mutates another flow's settings.
3. **A dry run makes no writes.** It does not create tables, advance offsets or reset checkpoints.
4. **Execution respects dependencies.** A failed producer blocks its consumers, not independent flows. Cancellation reaches active jobs.
5. **Refresh is explicit.** A materialized view recomputes. A streaming table resumes committed progress. A full refresh resets the output and its progress, subject to `pipelines.reset.allowed`.
6. **Progress survives sessions,** under the storage URI.
7. **Commit order is recoverable.** Recovery must tell an uncommitted attempt from a committed sink whose acknowledgment was lost.
8. **AutoCDC is stateful,** with ordering, deletes, tombstones and durable auxiliary state.

3 Architecture

3.1 Where SDP lives

SDP starts as a `pipelines` module inside `sail-spark-connect`. It is to be extracted into its own crate only when a second frontend needs it. The planner does not depend on Spark protobufs; requests are converted to Sail's `spec` types at the boundary (SAIL-SDP-DESIGN-DECISIONS.md §2).

The work reuses Sail's existing pieces: `PlanResolver`, `RelationScope`, the catalog manager and providers, `JobService`, task contexts and the actor model. It does not add a second scheduler or a new error framework. Sail's job scheduler still orders stages and tasks; the pipeline layer only orders flows.

3.2 Crates touched

The diff from `0ba425c03` to `fb3a6ab03` is 801 files, +143,358 / −2,229 lines. By changed-file count:

Area	Files	What changed
`sail-spark-connect`	154	The `pipelines` module: registration, SQL, analysis, materialization, execution, streaming, AutoCDC.
`python/pysail`	113	Pipelines test suite (about 70 files), ORC, TIME, catalog and Delta tests.
`sail-data-source`	91	ORC reader and writer; listing changes.
`sail-plan`	83	Relation scopes, catalog expressions, casts, error diagnostics.
`vendor/orc-rust`	77	Locally patched orc-rust 0.9.0.
`sail-execution`	54	Local job runner ownership, checkpoint effects, cluster stop, codecs.
`sail-common-datafusion`	41	`BoundedSource`, task resource scopes, managed streams, schema merge, structured errors.
`sail-sql-analyzer`	26	Pipeline statement lowering and source slices.
`sail-session`	18	Session lifecycle, cleanup records, shutdown, shared memory catalog.
`sail-delta-lake`	14	Delta streaming source, fenced commits.
`sail-sdp-control`	12	New: the CDC control model and Turso adapter.
`sail-iggy-bounded`	11	New: Iggy as a bounded source.
`sail-sql-parser`	12	Parser diagnostics and pipeline syntax.
Others	small	`sail-catalog-hms`, `sail-catalog`, `sail-function`, `sail-object-store`, `sail-iceberg` and more.

Two small Grust kernels are vendored: `vendor/grust-topology` (dense topological ordering with cycle witnesses) and `vendor/grust-columns`. No Grust or LakeCat service crate is imported, and there is no dependency on `lakecat-sail`, which would create a cycle (§2).

3.3 The `pipelines` module

Module	Role
`mod.rs`	Dispatches the six pipeline commands and drives the `StartRun` event stream.
`graph.rs`	`PipelineGraph`, `GraphDefinition`, `Output`, `Flow`; name parsing and qualification; registration and graph validation.
`sql.rs`	Prepares and commits a SQL file: parse, lower, resolve catalog expressions, register atomically.
`query_function_guard.rs`	Limits on eager work during definition; only `spark.sql()` is allowed inside a query function.
`analysis.rs`, `analysis/failure.rs`	Whole-graph analysis through `GraphScope`; unresolved-flow reporting.
`config/`	Flow configuration tracing, merging and resolution.
`schema*.rs`	Declared schemas, schema merge, the streaming-read schema contract.
`topology.rs`	Run order and cycle detection over the vendored topology kernel.
`selection.rs`	Refresh and full-refresh selection from `StartRun`.
`materialize.rs`	Table metadata materialization and persisted-view publication.
`reanalysis.rs`	Per-command input resolution, stopping at tables.
`execution.rs`	Batch execution, retries, events, sinks.
`append.rs`	Triggered append flows: checkpoints, file, Delta and bounded sources, tumbling windows.
`streaming/`	Admission, binding, lowering and the bounded-source glue.
`autocdc/`	AutoCDC definition, validation, the SCD1 processor, stateful operators, the durable fenced run.

3.4 Run phases

`StartRun` runs four phases. A dry run stops after analysis.

```
client (pyspark.pipelines / spark-pipelines)
  |
  | CreateDataflowGraph, DefineOutput, DefineFlow,
  | DefineSqlGraphElements                    (registration)
  v
+-------------------------------------------------------------+
| PipelineGraph (session-owned, keyed by a UUID graph ID)     |
|   outputs[], flows[]; no DDL, no catalog writes             |
+-------------------------------------------------------------+
```

```
  |
  | StartRun: selection checks, storage root check, start()
  v
+-------------------------------------------------------------+
| 1. ANALYSIS  analysis::analyze                              |
|    resolve every flow through GraphScope (no writes)        |
|    config fusion, validation in Spark's order, topology     |
|    -> AnalyzedGraph {order, dependencies, schemas,          |
|                      publications, accepted_confs}          |
+-------------------------------------------------------------+
  |                         dry run: stop here, "Run is COMPLETED."
  v
+-------------------------------------------------------------+
| 2. ADMISSION  AutoCDC graphs: autocdc::admitted,            |
|    CdcRun::prepare (targets, sources, control store)        |
+-------------------------------------------------------------+
  v
+-------------------------------------------------------------+
| 3. MATERIALIZATION  materialize::tables, materialize::views |
|    create/merge table metadata, then publish persisted views |
+-------------------------------------------------------------+
  v
+-------------------------------------------------------------+
| 4. EXECUTION  in topological order, one flow at a time      |
|    materialized view -> reanalysis + batch write (retries)  |
|    table / sink      -> append::run (microbatches, ckpt)    |
|    AutoCDC table     -> CdcRun::run_flow (fenced SCD1)      |
+-------------------------------------------------------------+
  |
  v  PipelineEventResult messages ... "Run is COMPLETED."
client
```

The event channel is drained while the run future is polled. A `watch` channel on the graph carries cancellation. When it fires, the run future is dropped and the stream ends with "pipeline run cancelled".

4 Components

4.1 Graph registration and identifiers

`handle_command` in `mod.rs` dispatches each command against the session's `SparkSession`.

Graph IDs. `CreateDataflowGraph` creates a `PipelineGraph` and stores it under a fresh `uuid::Uuid::new_v4()`. The ID is session-owned and ephemeral. Nothing persistent is keyed by it; checkpoints and control records use qualified dataset and flow names (§3, DESIGN.md §1). The command also sets the session's default catalog and database from the request, if given.

Names. `graph.rs` parses every name with Sail's SQL object-name parser, so a quoted identifier such as `` `a.b` `` stays one part. Names are then lowercased with ASCII rules. `GraphDefinition::qualify` turns one or two parts into `catalog.database.table`. A one-part name with an unset database is rejected with an analysis error, not a panic. Four or more parts give `REQUIRES_SINGLE_PART_NAMESPACE`. Temporary views and sinks must be single-part (`TEMP_VIEW_NAME_TOO_MANY_NAME_PARTS`). The response carries a `ResolvedIdentifier` with catalog, namespace and table.

Flows. `define_flow` checks identifier arity first. A multipart flow name that differs from its target raises a raw `MULTIPART_FLOW_NAME_NOT_SUPPORTED`. A flow targeting a temporary view or sink stays unqualified. The flow's SQL config is the graph defaults overlaid by the flow's own settings; this overlay is not itself a conflict. Any `DefineFlow.once`, even `false`, is rejected with `DEFINE_FLOW_ONCE_OPTION_NOT_SUPPORTED` (0A000) before graph lookup, as Spark's Connect handler does. AutoCDC flow details are validated at definition time by `autocdc::definition`.

Idle check and run entry. Every definition command first calls `ensure_idle`, which refuses changes while a run is active. `start` validates registration and refuses a second run on the same graph with `SPARK_CONNECT_ILLEGAL_STATE.EXECUTION_STATE_EXECUTE_HOLDER_ALREADY_EXISTS_GRAPH` (XXSC0). Spark retains the execution entry after a run, so a refresh registers a new graph.

StartRun preconditions. Selection options are validated before the execution entry is reserved, so a bad option can be corrected on the same graph. Storage must be present, and `validate_storage_root` requires an absolute path with a URI scheme, else `PIPELINE_STORAGE_ROOT_INVALID` (42K03).

Query functions. PySpark marks definition requests with a `PipelineAnalysisContext` user-context extension. `query_function_guard.rs` reads it. Inside a query function only `spark.sql()` may reach the server. AnalyzePlan is rejected, and so is a DDL-string schema, because parsing it would be an analysis RPC (§3, CF-0569). Side-effecting statements in any definition request are rejected.

Direct dataset statements. Outside a pipeline, `CREATE MATERIALIZED VIEW` or `CREATE STREAMING TABLE` gives `UNSUPPORTED_FEATURE.CREATE_PIPELINE_DATASET_QUERY_EXECUTION`, and `CREATE FLOW` gives `CREATE_FLOW_QUERY_EXECUTION`. Sail rejects before the INSERT child runs; Spark writes the child first. That is a documented difference.

Identifier policy. Sail keeps its Unicode unquoted-identifier policy. It accepts dotted and hyphenated table names such as `output.with.dot` and `hyphen-mv`, which Spark's session catalog refuses with `INVALID_SCHEMA_OR_RELATION_NAME`. This is engine-wide, not pipeline-specific. Temporary views accept `v`, `SESSION.v` and `SYSTEM.SESSION.v`; any other qualifier is `INVALID_TEMP_OBJ_QUALIFIER`.

4.2 SQL pipeline files

A SQL pipeline file arrives as one `DefineSqlGraphElements` command with the file text and path.

Parsing and lowering. `sail-sql-analyzer/src/pipeline.rs` lowers pipeline statements using the ordinary SQL grammar, in an explicit entry point that never executes DDL. `sql::prepare` parses the whole file into statements with their lexer spans, then lowers each statement. Lowering stops at the first failure, but preparation itself owns no graph state.

Source slices. A persisted view stores its original query text, as Spark's `originalText` does. Sail takes it as a slice of the source using lexer spans: from the first query token to the last, with interior whitespace, comments, hints and quoted text exact. There is no regex and no reconstructed SQL. Spans are byte offsets, because the Chumsky lexer advances by `char::len_utf8()`; this was pinned for multibyte literals and Unicode identifiers (SQL-SOURCE-DESIGN.md).

Preparation outside the lock. `SET CATALOG` takes an expression. Lowering produces a `SetCurrentCatalogExpression` command, and preparation resolves it through `PlanResolver::resolve_catalog_name`, which may need catalog access. That is asynchronous. So `sql::register` checks the graph is idle, drops the lock, awaits preparation, then takes the lock again to commit. The blocking mutex is never held across an await and was not replaced by a Tokio mutex (§3). Catalog folding uses the session SQL config, not file `SET` or graph defaults, as Spark does. Three constant

policies are kept distinct: bare `SET CATALOG` optimizes first, a wrapped `IDENTIFIER` evaluates during analysis, and the SDP bare directive runs analysis checks before evaluating (CATALOG-EXPRESSION-DESIGN.md).

File atomicity. `PreparedSql::commit` clones the current graph into a draft, registers each statement in order, and replaces the graph only if every statement succeeds. After the file, the draft's catalog, database and SQL config are reset to the graph's values, so `USE` and `SET` are local to the file. A failure produces `PIPELINE_SQL_GRAPH_ELEMENT_REGISTRATION_ERROR` (42000) with the offending statement text and `file:line`. Spark keeps definitions registered before the failing statement. Sail registers none. The standard client aborts the run in both cases, and four Sail tests pin the all-or-nothing behavior.

Directives. Only `USE SCHEMA` and `USE CATALOG` are pipeline directives. A bare `USE x` is rejected as unsupported, as Spark 4.2.1 does; this was measured on Spark (§3).

Statement checks. Several rules follow measured Spark behavior:

- Column constraints give `_LEGACY_ERROR_TEMP_0035`, checked before other dataset restrictions, with no SQLSTATE and no file name.
- `INSERT ... IF NOT EXISTS` is `INVALID_STATEMENT_OR_CLAUSE`.
- Flow INSERT checks run in the order partition, columns, overwrite, by-name.
- CTAS and RTAS are rejected with Spark's node names.
- An invalid partition transform gives Spark's "Invalid partitioning transform".
- Declared column names on a view are ignored, and the view comment is kept.
- `REPLACE WHERE` and `BY NAME` go through `TruncateIf` and `ByName`. Ordinary `REPLACE USING` execution stays unsupported, with no append approximation.

Parser diagnostics. Syntax errors come from a generic variant-level hook in the parser with structured diagnostics and UTF-16 column positions. The renderer prints nothing rather than guess when the expected set is empty or end of input is expected. Post-parse filters, token regexes and a global map from parser strings to Spark errors were rejected (SQL-SYNTAX-DIAGNOSTICS-DESIGN.md).

IDENTIFIER and parameters. Literal `IDENTIFIER('...')` strings normalize to named reads and become graph dependencies. A computed target is resolved late, through a live catalog lookup, and is never a graph dependency. SQL parameters freeze input identity at registration (DYNAMIC-REFERENCE-BINDING-DESIGN.md).

4.3 Analysis and validation order

`analysis::analyze` resolves every flow before anything is written. There is no eager analysis at `DefineFlow`.

GraphScope. Each flow is resolved by Sail's ordinary `PlanResolver` with a request-local `RelationScope`, `GraphScope`. All declared outputs are known before analysis begins, so an unresolved local name never falls through to a stale table from an earlier run. Views resolve to their analyzed plans. Tables and materialized views resolve to *virtual tables*: an empty input with the output's schema. Analysis never needs to read a producer's data. `RelationScope` was extended in `sail-plan/src/resolver/scope.rs` to keep the requested read mode, observe external source reads and their options, and resolve late-bound IDENTIFIER targets in a separate phase.

Fixpoint loop. Analysis iterates over outputs until every output is resolved or no progress is made. A flow can be `Resolved`, `Pending` (waiting on an unresolved input) or `Failed`. If a round makes no progress, the pending flows become failures with "Failed to read dataset '…'. Dataset is defined in the pipeline but could not be resolved." Failures are reported through an `UnresolvedPipeline` wrapper with typed direct and

downstream maps; the client sees an AnalysisException. Failure events are emitted with the downstream group first, then the direct group, each sorted. Scala's map order is unspecified, so Sail's sort is a disclosed divergence.

Lazy virtual schemas. A virtual table is constructed with its flows' plans, not its schema. An unread output must not fail inference ahead of another unresolved query. There is no fallback schema and no collapsed error (CF-0553, CF-0559).

Validation order. After all flows resolve, validation follows Spark's order (CF-0082, CF-0091):

1. user-specified schemas;
2. topology;
3. CDC multiflow, then materialized-view multiflow;
4. `INVALID_TEMP_OBJ_REFERENCE` (a persisted view reading a temporary view);
5. reset policy;
6. streamingness, and temporary-view CDC;
7. the phase-9 raw schema merge.

The no-flow check covers tables and views only; sinks are excluded, as in Spark. Among destinations with several flows, any non-streaming flow gives `MATERIALIZED_VIEW_WITH_MULTIPLE_QUERIES`. The phase-9 merge error is a SparkException `CANNOT_MERGE_INCOMPATIBLE_DATA_TYPE` (42825), with left and right in Spark's order. A table with any failed producer is never installed.

Repeated columns. A query that produces the same column twice passes analysis and a dry run; its flow fails at run time with `AMBIGUOUS_REFERENCE`. Analysis renames repeats internally (`__sail_duplicate_column_<i>`) so the plan has a schema, and records the column. A view that repeats a column is still refused during analysis, where Spark accepts `SELECT *` over it and refuses a reference. That is a known difference (batch-gaps).

Streaming read schema. A streaming read of a graph dataset exposes the schema of Spark's `MemoryStream` encoder, not the input schema. `schema_stream.rs` is the single definition of that contract: field metadata dropped at every depth, intervals widened to their families, TIME widened to TIME(6), and CHAR and VARCHAR refused during the consumer's analysis. Both lanes use the same function (CF-0566, measured on Spark).

Output. Analysis exports an `AnalyzedGraph`: topological order, dependencies, schemas, persisted-view publications (`CreateViewOptions`), accepted configurations by flow, and repeated columns. It never exports plans or providers. `topology.rs` orders datasets with the vendored Grust kernel and reports a cycle between datasets, with a flow standing for its target.

4.4 Flow configuration fusion

Spark analyzes each flow under a private clone of the session SQL config. FLOW-CONFIGURATION-DESIGN.md records the exact behavior Sail reproduces:

1. The flow's declared map is the graph defaults overlaid by its own settings.
2. The first semantic pass applies the declared map, and each upstream *view* read during input substitution contributes its own resolved map, in Catalyst's traversal order. Tables contribute nothing.
3. Only after that pass succeeds, the flow's map and its inputs' maps are unioned. A key with different values is `DUPLICATE_FLOW_SQL_CONF`. Sail reports the smallest conflicting key (CF-0282).

4. If the union differs from the declared map, the query is resolved again under the union. That second map is the flow's accepted configuration. A view's accepted map includes its upstream union, so consumers inherit it transitively.

The `config/` module splits this into `trace.rs` (an ordered trace of graph inputs before native resolution), `merge.rs` (pure merge with provenance per flow) and `resolve.rs` (the two passes). Shortcuts were measured and rejected: a semantic first pass used only to discover reads can fail before reaching the input whose setting makes the query resolvable, and nested projections and scalar subqueries need Catalyst's input substitution order. CTEs bind in definition order, so even an unused CTE definition can contribute settings.

Per-flow config is applied as a `PlanConfig` override (`plan_config_with_overrides`). The shared session is never mutated (CF-0128, CF-0131). The thirteen public differential probes all match Spark's error-condition and success-row outcomes.

4.5 Materialization and catalog publication

A real run materializes metadata, then publishes views, then runs flows. A failed write may leave tables behind, as in Spark, which does not preflight publication as one atomic step (DESIGN.md §3).

Tables. `materialize::tables` creates or updates the metadata of every selected table before any flow runs, matching Spark's `DatasetManager`. Rules (METADATA-TABLE-MATERIALIZATION-DESIGN.md):

- Inferred schemas become nullable; declared schemas stay exact.
- An ordinary refresh of an existing table merges with the existing schema on the left, without truncation.
- Materialized views keep replace and truncate.
- A full refresh of a streaming table is refused before any effect unless `pipelines.reset.allowed=false`, which downgrades it to an ordinary refresh as in `DatasetManager.constructFullRefreshSet`. The whole-graph check is labelled a temporary Sail policy.
- Partition columns in file tables move last.
- A missing partition column still creates the table; each flow attempt then fails with `COLUMN_NOT_DEFINED_IN_TABLE`, and independent flows run. Such failures are *deferred* and reported per output at attempt time.
- A fully refreshed AutoCDC target's auxiliary table is dropped with `DROP TABLE IF EXISTS`, as Spark does.

Views. `materialize::views` publishes every persisted view from its analyzed definition, regardless of table selection; an excluded upstream table need not exist yet. Spark's `verifyAutoGeneratedAliasesNotExists` is reproduced: a persisted view must name each expression column. A persisted view is never silently materialized as a table.

Through providers. Outputs are published through the existing catalog provider operations. The public `execute()` path rejects Table and Sink outputs before any catalog write; metadata-only materialization uses a private native entry point (CF-0440).

Caches. `REFRESH TABLE` and `spark.catalog.refreshTable` clear every file-listing and statistics cache entry, since those caches are not keyed by table. `TRUNCATE` invalidates the list-files cache.

4.6 Batch execution

`execution::execute` walks the analyzed order. It runs one flow at a time.

Events. For each runnable output, each flow is first reported `Flow <name> is QUEUED.` or `Flow '<name>' is EXCLUDED.` by the selection. Each attempt emits `PLANNING` (materialized views only), `STARTING`, `RUNNING`, then `has COMPLETED.` or `has FAILED.` with the error. The texts follow Spark's `FlowProgressEventLogger`. The run ends with `Run is COMPLETED.`

Failure propagation. A failed flow marks its target failed. An output that reads a failed output, directly or through views, is skipped with `Flow '<name>' SKIPPED due to upstream failure(s).` Independent flows still run. The first failure becomes the run's error.

Retries. The retry count is `spark.sql.pipelines.maxFlowRetryAttempts` from the flow's accepted config, default 2, as in `TriggeredGraphExecution`. A materialized view retry truncates the target first, because a failed attempt may have left files. A streaming flow that exhausts its retries emits Spark's "has FAILED more than N times and will not be restarted" message. A flow whose streaming sources changed is not retried and asks for a full refresh.

Reanalysis. Each attempt resolves its inputs afresh through `reanalysis::resolve_inputs`, under the flow's accepted config. Views are traversed through their flows; only tables stop reanalysis. Each table command resolves its own upstream map; the earlier run-wide plan cache was removed. Producer queries are not inlined, and persisted views were resolved once during analysis (BATCH-PROVIDER-MIGRATION-DESIGN.md).

Write commands. A materialized view becomes an ordinary Sail write command resolved with the graph scope and run through `JobService`. Tables and sinks go to the append path. A `memory` sink writes Parquet under the storage root and is exposed as a temporary view named after the flow, which is how a session reads Spark's memory sink.

Writes through `spark.sql()`. A write returns no rows. CTAS returns an empty result, as Delta on Spark does. The write is recognized by its single UInt64 `count` column.

4.7 Errors and Spark fidelity

The client tests compare error class, condition, SQLSTATE and parameters, so error fidelity is a large part of the work (EXECUTION-ERROR-DESIGN.md).

- `StructuredSparkError` lives in `sail-common-datafusion`, with a structured `CommonErrorCause` carried through serde and `RemoteError`, so a worker error reaches the client with its class intact. There is no new error framework. Connect prefix matching and pipeline-only repairs were rejected.
- Catalog analysis failures get the registration wrapper (42000). Execution, arithmetic and ANSI cast failures stay raw.
- Division and remainder by zero are typed UDFs guarded as `dividend / spark_zero_divisor(divisor)`, raising `DIVIDE_BY_ZERO` or `REMAINDER_BY_ZERO`. Lowering never evaluates.
- A legacy string-to-float cast goes through TryCast; one `SparkStringToDecimal` UDF serves ANSI, legacy and `TRY_CAST`; an ANSI string-to-numeric or boolean cast captures the failing value only on the failure path.
- Which row's error surfaces must match Spark (CF-0355).
- Unresolved-column and ambiguity errors are fixed in general Sail, not translated in the pipeline layer.
- `CATALOG_NOT_FOUND` is raised only at explicit catalog loads.
- A missing data source gives Spark's `DATA_SOURCE_NOT_FOUND` text. On Spark the client sees it wrapped in `TableMaterializationException` inside an `UnknownException`; that class difference is disclosed.
- A released session returns `[INVALID_HANDLE.SESSION_CLOSED]` on every RPC, with Spark's text, class and parameters.

- An unknown commit outcome stays nonretryable. A commit failure followed by a cleanup failure gives a typed `DataFusionError::Collection` that keeps both.

Fixing these in general Sail moved some non-pipeline behavior. Under ANSI, a string cast now lowers to a guarded function, which first broke decorrelation of correlated scalar subqueries in 56 tests. That regression was found only by comparing against upstream and was fixed (chapter 5). TPC-H and TPC-DS plan snapshots changed in 38 places, all plan text, with operator counts kept except 8 fewer projections.

4.8 Streaming

Model. Streaming runs as bounded AvailableNow microbatches over ordinary Sail batch plans. A run captures a frontier once, reads to it in one or more batches, and stops. Extending Sail's continuous `StreamSource` path is deferred (§13).

Admission. `streaming/admission.rs`, `binding.rs` and `lower.rs` decide what an append flow may run. A batch intent is the only authority. Rejection is exhaustive and happens before resolution. Joins follow Spark's `UnsupportedOperationChecker`, with no stream-stream joins in the append path. Any non-immutable function makes a plan non-replayable. Mixed read modes and dynamic Python UDTFs are refused. A streaming read of a source that cannot be read is refused, never treated as success. A union of streaming and batch inputs is refused, as Spark does, since `609a6c251`.

Append flows and checkpoints. `append.rs` runs a streaming table or sink fed by streaming reads. Progress lives under `<storage>/_checkpoints/<catalog>/<db>/<table>/<flow>/<n>`, and a full refresh starts directory `n + 1`. A batch's files are recorded before it is written and marked read after it, so a crash between the two replays exactly that batch. For a plain file table that is at-least-once: there is no idempotent commit, so a replay can duplicate rows.

- **Files.** A streaming read of a file-backed table sees files this flow has not read, as Spark's file stream source does. Replaced or deleted files are not revisited, so `INSERT OVERWRITE` on a source adds only its new rows. Batch reads in the same flow see the latest data.
- **Delta.** A Delta source is read from its log (`sail-delta-lake/src/stream.rs`), with a version as progress. A fresh flow starts from a snapshot of the latest version, or from `startingVersion` or `startingTimestamp`. After that it tails `Add` actions. Rules measured on Spark 4.2.1 with Delta 4.4.1: a commit that removes data fails the read unless `skipChangeCommits`, `ignoreChanges` or `ignoreDeletes` allows it; commits without data change, such as `OPTIMIZE`, contribute nothing; later commits must be readable with the latest schema. Rate-limit options are validated but have no effect, since one run reads to the end of the log.
- **Bounded sources.** A `BoundedSource` is read as exact ranges from each partition's cursor, a bounded number of rows at a time, so one run writes several batches up to the frontier it captured.
- **Windows.** The only stateful append query is a tumbling-window aggregate with a watermark. Open windows are held under the checkpoint until the watermark passes their end, then aggregated once. The shared windowing code (`Windowed` with `slide_micros`, `offset` and a list of watermarks; `event_window`) is used by AutoCDC too. The append path refuses sliding windows, more than one watermark, and a watermark not directly on a streaming read.

BoundedSource. The trait lives in `sail-common-datafusion/src/streaming/bounded.rs`. It was moved there unchanged from the Connect module so adapters in lower crates can implement it without a dependency cycle. It is protocol-neutral. An adapter describes partitions with immutable *incarnations* and opaque *cursors*, plans a range end in its own terms, and scans exactly `[start, end)` or fails closed. Generic code stores, compares for equality and serializes cursors; it never parses them or does arithmetic on them. `ScanError` covers retention gaps, recreated partitions, non-consecutive positions, missing partitions and

foreign cursors. `SourceError` separates deterministic refusals from `Unavailable`, the one class a control loop may retry with the identical intent.

Iggy. `sail-iggy-bounded` implements `BoundedSource` for one Apache Iggy stream and topic, against the Iggy SDK at a pinned revision. Its rules come from behavior recorded on the pinned broker: a poll past retained data silently resumes at the first retained offset, and a recreated partition reuses its ID with offsets from zero. Neither is a broker error, so every read checks the offsets and incarnation it got back. The first-offset check is the only gap detector; a gap or an incarnation change fails closed. Reads never acknowledge: AutoCommit is off and no consumer position is stored, so a failed read can be repeated unchanged. Any message that does not decode fails the whole range. The `iggy` format gives planning a schema only and refuses batch scans. Decoding and the archival envelope are ported from the private `sail-iggy` reader at `5816b9b`; the mirror's private SessionContext and planner, its `RepartitionExec` stripping and its version-as-offset comparisons were not ported (STREAMING-REUSE.md, §13). Spark has no Iggy source, so these tests state Sail's own contract.

4.9 AutoCDC

AutoCDC applies a change feed to a target table. Each row carries keys, a sequencing value and possibly a delete marker. SCD Type 1 keeps the latest row per key.

Definition. `autocdc/mod.rs` ports the `ChangeArgs` contract, the `PipelinesHandler.buildAutoCdcFlow` checks, the schema validators of `AutoCdcMergeFlow` and the AutoCDC graph validations. SCD2 is rejected faithfully with `AUTOCDC_SCD2_NOT_SUPPORTED`. The flow's query is a streaming read of its source, resolved against the graph, so a dry run analyzes it like Spark's `AutoCdcFlow`. Name parsing is CDC-only, lowercasing follows Java `Locale.ROOT` rules (ported from JDK 17 tables), and key lookup reproduces Scala's map order (§14).

Admission. `autocdc::admitted` accepts a graph only if every AutoCDC flow is SCD1 into a Table output. Otherwise execution refuses with "incremental pipeline execution" before any write. `CdcRun::prepare` then creates missing targets, opens the control store and resolves each flow's sources. AutoCDC targets are removed from the plain batch selection, because their writes go through the authority.

The SCD1 processor. The pure components follow Spark's `Scd1BatchProcessor` and `Scd1ForeachBatchHandler`:

1. `microbatch.rs`: validate the original rows (orderable sequencing type, no null sequencing value, no null key) before anything else, so an invalid row cannot disappear because another row with the same key wins.
2. `dedup.rs`: keep the latest event per key with `max_by(struct, sequencing)`, using Sail's `MaxByFunction`. A tie keeps the last row; Spark leaves ties undefined.
3. `reconcile.rs`: reconcile the batch against the auxiliary state.
4. `merge.rs`: two `MERGE INTO` commands through Sail's MERGE resolver. The first merges keys and the sequence into the auxiliary table `__spark_autocdc_aux_state_<target>`, which keeps tombstones: a matched tombstone advances on a strictly newer delete and is removed by an upsert at or above its sequence. The second merges the reconciled rows into the target: a newer delete deletes, a newer upsert updates every non-key column, an unmatched non-delete inserts.

The auxiliary table records the SCD type and the key column names as reserved properties. Key drift is kept distinct from sequence evolution. Spark's `createAuxiliaryTableIfNotExists` is reproduced in `write.rs`.

State between batches. Spark's state store keeps operator state across microbatches. Sail now does the same for stateful queries before the merge (`stateful.rs`, `state.rs`), replacing an earlier evaluation-by-difference approach:

- A **watermarked aggregate** keeps the rows of windows still open and the watermark. Sliding windows and window start times are covered.
- A **deduplication** keeps the keys it has seen.
- An **inner join of two streams** keeps both sides; a pair is emitted once, when its later row arrives.

The query watermark is the minimum over the streams, with a stream that has no rows counting as 0. Only the query watermark survives a restart, as in Spark. Join and deduplication state is append-only: each batch writes one segment, plus a manifest that lists every segment, so a batch's cost does not grow with the state. Snapshots are named by the progress they follow, so a replayed batch reads the snapshot it started from. Writes go to a sibling file and are renamed.

Flow sources. `flow_source.rs` turns a flow's query into one bounded source. Each streaming read under the query, including reads through views, is a leaf. Supported leaves include files (listed in first-seen order in a list that only grows), Delta table logs, Spark's rate source, and registered bounded sources such as Iggy. The batch's rows are archived before its first effect, so a replay never re-evaluates the query.

The control store. CDC-CONTROL-STORE-DESIGN.md is the specification, written by the batch lane and built by the CDC lane. The central decision: one transactional SQL authority writes the target guard, the catalog-ratified sink head and the effect receipt *in a single transaction*. A lease in one system followed by a Delta commit in another cannot fence a stale writer. Rejected alternatives were a separate compare-and-set or transaction ID as the fence, a Delta control table, an advisory lock with a local file, a `PutMode::Create` JSON store, and Delta DomainMetadata or table properties.

`sail-sdp-control` holds it:

- `identity.rs`, `records.rs`, `digest.rs`: identities, fence tokens (authority incarnation, target key, reset generation, owner epoch, owner ID), the durable records, and canonical SHA-256 request digests.
- `transitions.rs`: pure guarded transitions with typed outcomes (`Conflict`, `Fenced`, `IdentityMismatch`, `PreparedMismatch`, `SourceUnavailable`, `StateUnknown`, `EvidenceUnavailable` and others). None of them authorizes a rerun.
- `control.rs`: the operations (enroll, acquire and renew, seal intent, prepare, ratify data or no-op, lookup, complete batch, begin and finalize reset, acknowledgments), each one transaction over the `Store` seam. The body has no network or object-store I/O and reruns on a fresh snapshot after a lost write race. A failed commit that is not a lost race is reported as unknown, never as absent.
- `memory.rs`: an in-process store for model tests that states it is not an authority.
- `turso_store.rs`: the Turso adapter, one `BEGIN CONCURRENT` transaction per operation on a database in MVCC mode. Its stated qualification excludes power loss, a file shared by live processes, and restore or fork. One authority process owns a database file.

At run time `CdcRun` opens the authority at `<state root>/_control/authority.db`. For a remote warehouse the state root is local: `SAIL_AUTOCDC_STATE_DIR` or `~/.sail/autocdc/<sha256 of the warehouse URL>`, with a marker `_autocdc_state_owner` written in the warehouse. A warehouse that has the marker but no local state stops the run instead of starting over against tables with history. The lease is 60 seconds and is renewed each batch. The microbatch bound is 10,000 rows per partition.

Fenced and plain targets. A Delta target whose log the authority can watch, and that is not catalog-managed, is fenced: `DeltaCommitExec` calls the authority to ratify the staged entry (`autocdc/authority.rs`,

sail-delta-lake/src/fenced.rs). The existing Delta commit and ratification path is kept; there is no second CDC writer. An Iceberg table, or a Delta table whose catalog ratifies commits, already orders its commits and takes the plain path.

The two-table protocol. durable.rs runs each batch through these states:

```
IntentSealed -> Prepared -> AuxConfirmed -> TargetConfirmed
             -> ProgressCommitted -> AckPending -> AckConfirmed
```

A batch is sealed with its source range before its first effect. Its reconciled rows are archived, and an archive with another digest stops the run, never reruns it. Each merge is a fenced effect with a stable (intent, step) identity, so a retry under a new request ID returns the original receipt, never a second effect. Completion checks both receipts and advances progress to the sealed end in one transaction, then queues the source acknowledgment. The auxiliary and target heads are not claimed to be one atomic snapshot. One target-wide slot spans flows and pipelines, so no other CDC flow or mutation interleaves with an unfinished batch.

Initialization and reset. Target enrollment and auxiliary-table creation use a short Initialization intent in the same slot, with fenced receipts but no progress. A full refresh drops the auxiliary table, as Spark does, and the authority, not directory listing, reserves the next checkpoint number n. reset.allowed=false keeps Spark's downgrade to an ordinary refresh. Enrolled tables refuse ALTER. Checkpoints are laid out as warehouse/_checkpoints/<destination>/<flow>/<n>, mirroring Spark.

Crash recovery. A successor resumes an unfinished batch from what the authority recorded and repeats only effects that have no receipt. A recovery run waits out a dead owner's lease rather than failing. The durable run exposes crash points (for example, after BeginReset, after truncating the target, during initialization) for tests. The gate's crash.py seeds kill the process (SIGKILL) at such points; on the CDC gate every recovery was exact. Only process death is qualified. Power loss and restore are an accepted limit (user decision 19).

Two runs. durable.rs runs a fenced Delta target under the control store. run.rs::run_plain_flow runs a target whose catalog or format already orders its commits (Iceberg, catalog-managed Delta such as Unity). There the checkpoint is the progress, as in Spark, and a replayed batch's merge is safe because rows win by sequence. A plain reset drops and recreates the target and its auxiliary table.

4.10 Session lifecycle, shutdown and effect ownership

A pipeline run writes checkpoints and tables under prefixes that session cleanup deletes. The baseline had races where a deleted checkpoint object could reappear. The runtime baseline in SESSION-CLEANUP-LIFECYCLE-DESIGN.md observed it directly: a held multipart completion republished a deleted checkpoint object after its caller was canceled and joined.

Session states. Lifecycle policy lives in sail-session, with states Creating, Running, Deleting, Deletion-Failed, Deleted and Failed. There is no resurrection. A deleted session keeps a tombstone; releasing a never-seen session is a no-op, as Spark's ReleaseSession handler does.

One cleanup record. session_manager/actor/cleanup.rs defines a single record that owns the end of a session. Explicit deletion, idle deletion, session failure and manager shutdown all share it, so a session cannot be cleaned up twice. The outcome is published by the driver and observed outside the actor. RPC callers are observers only. A dropped driver reports interruption, never success. A failed cleanup keeps its resources reachable.

Manager shutdown. The manager goes Open, then Closing, then terminal (shutdown.rs). Closing seals session admission but keeps the actor answering driver lookups, which the gateway needs while it stops.

`serve.rs` starts manager shutdown at the same time as transport draining, not after it: an active response can wait on a job that only the manager can stop. The actor system is always joined, and neither the transport failure nor the cleanup failure hides the other.

Local stop. Stop seals admission, requests cancellation of every job before awaiting any, and returns a result that can fail. Effects are joined before the namespace is deleted. Stop failures are sticky.

Effect ownership. Several primitives in `sail-common-datafusion` and `sail-execution` make effects visible to stop:

- `TaskResourceScope` (`task_resources.rs`) owns resource supervisors for one execution scope. `seal` blocks new supervisors without canceling; `cancel` latches; Drop only requests cancellation. A supervisor reports success only after completion or cleanup is confirmed.
- `managed_stream.rs` gives the consumer an exclusive poll lease, with no forwarding task and no prefetch.
- `OwnedObjectStore` (`owned_store.rs`) attaches storage operations to an existing owner. `checkpoint_effects.rs` routes checkpoint writers through it, so stopping a job waits for its PUTs and multipart uploads. One multipart supervisor owns create, parts, complete and abort; there is no blind abort after complete was polled.
- A move-only preparation lease admits planning before any resolver or provider work (PRE-PLANNING-OWNERSHIP-DESIGN.md).

Settled failures. A job failure already delivered to its query, with its resources drained, is logged at shutdown and does not fail cleanup. An unknown commit outcome, or resources that did not drain, still fail shutdown (CF-0606, `b1f41c6bf`).

Interrupting a run. Interrupting a run, or releasing its session during the run, stops the flows. Spark cancels the operation but lets the flows finish. Sail stops them because nothing else could stop such a run, and shutdown needs the flows stopped. This is a strict expected failure.

Not done. Fallible actor and gateway results need the common actor framework. Non-checkpoint table writers are not routed through the effect owner. Cluster worker effects are not qualified.

4.11 Cluster mode

The pipeline suite also runs with Sail's local-cluster mode (driver plus workers). Racy tests found real defects:

- **Driver limit.** A session released while workers start could hang ReleaseSession indefinitely: a worker could stick in its own shutdown while its heartbeat ran on, and the driver joined it forever. A stopping driver now gives up on its workers after 30 seconds and aborts what remains (`a4dfce84a`). Stop retries for unreachable workers were added.
- **Worker limit.** The worker's gRPC graceful stop is what held it. It is bounded at 5 seconds, then aborted (`2e73a0b87`).
- **Canceled jobs.** A job canceled by a stopping driver ends its output with an error, never a clean end. A refused retry reports the failure that caused it.
- **Checkpoint cleanup.** Clustered cleanup cannot confirm that remote writers stopped. It returns an explicit unconfirmed-cleanup failure and keeps the prefix. `ClusterJobRunner.stop` returning Ok does not authorize deletion. Releasing a cluster session that used checkpoints therefore fails, keeps the files, and the server exits 1 (user decision 2).

The remaining cluster issue is open: a canceled job's stage-0 Flight `do_get` stream stays open after its tasks are aborted, which holds the worker's graceful stop until the 5-second abort (found in 2 of 3 firings).

4.12 ORC

ORC was needed because Spark's pipeline suites read and write it. All ORC semantics live in Sail's shared data-source path (`sail-data-source/src/formats/orc/`); LakeCat needs no ORC API.

Dependency. Sail uses orc-rust 0.9.0 core directly with a small adapter and `default-features=false`, not the `datafusion-orc` adapter, and without upgrading Sail's prost for it. The crate is vendored at `vendor/orc-rust` as a locally patched upstream package; `SOURCE.json` records the upstream tag and each patch (bounded compression blocks, validated footer ranges, a decoded-output budget, validated schema trees, and writer admission).

Reading. A stripe belongs to the half-open byte range containing its start. Tail and footer budgets are 64 MiB each, and schema depth is limited to 64. Unsupported Hive and forced-positional read modes are rejected at the read boundary. Source-read case sensitivity has a seam that applies to ORC only (ORC-SOURCE-DESIGN.md).

Writing. One stateful native `ArrowWriter` per file, with one footer, and a retained abortable upload handle. There is no per-batch concatenation, no `Vec` buffering, no text serializer and no gzip wrapper. Unsupported schemas are rejected before any byte is written; the native builder writes magic bytes before constructing encoders, so the check must come first (ORC-WRITER-DESIGN.md). TIMESTAMP, TIMESTAMP_NTZ, DECIMAL, ARRAY, MAP and STRUCT are written, and native encoder memory is reserved per slice. Compression precedence is `compression`, then `orc.compress`, then the session codec; the default is zstd, as in Spark. `test_orc_write.py`: 74 passed on Sail and on Spark; its negative control failed 45.

Timestamps. TIMESTAMP is stored as `TIMESTAMP_INSTANT`. Spark's stored kind depends on the writer JVM's time zone, which a Sail worker cannot know. An instant inside ORC-763's window, about one second before the epoch, fails the write. Spark writes it and reads back a value one second off (user decision 3).

Withdrawn. A candidate applied graph and flow ORC overrides as one blanket snapshot. Spark applies them phase by phase, so settings are resolved per phase.

Limits. No hybrid-calendar rebase before the Gregorian cutover. Spark on a non-UTC JVM reads Sail's instants before 1883-11-19 with local-mean-time drift. Memory reservations are estimates and do not bound the heap. Not run on real object stores.

4.13 TIME type

Spark 4.2.1 has a `TIME(p)` type with precision 0–6. Pipelines need it for schemas and the streaming-read contract. It is a shared type-system change, not a pipeline patch (TIME-SEMANTICS.md).

- `spec::DataType::Time { precision }`, default 6, stored as Arrow `Time64(Nanosecond)`, with the declared precision in hidden `SAIL::spark::time` field metadata.
- Schema merge accepts equal precisions with merged nullability and rejects different precisions recursively. No widening.
- A cast to a different precision `q` truncates to `10^(9-q)` nanoseconds, even when widening. A same-precision cast keeps the bits. The rule recurses into arrays, struct fields and map keys and values. The transform runs before Sail's positional rename and leaf casts, and is carried through the worker codec.
- Connect accepts every precision 0–6 and preserves nanoseconds in non-null literals.

Eight shared TIME checks pass unchanged on Spark 4.2.1 and on Sail; the previous Sail binary passed one. Remaining shared work: SQL type and literal lowering, TIME string parsing, configuration admission and field-aware schema converters.

4.14 Catalogs

Catalog	Coverage and notes
Memory	Default. Fixtures expect a fresh catalog per session.
Shared memory	`catalog.shared_memory`, off by default. When on, what one session creates is visible to others, as in Spark; temporary views stay per session. Turning it on by default broke 104 tests. The CLI tests run with it on in every gate.
Second catalog	A server with two memory catalogs (`run-second-catalog.sh`). Both SqlPipelineSuite catalog-crossing cases are ported. 13 passed.
Hive metastore	Standalone Hive 4.0.1 with embedded Derby, no Docker (`run-hive-metastore.sh`, downloads pinned by SHA-1). 69 passed, 2 xfailed.
Unity Catalog	v0.4.0 built from source and run locally, via `PYSAIL_UNITY_CATALOG_URL`. 230 passed on the CDC gate, including AutoCDC into Unity.
Iceberg	86 passed on the CDC gate. AutoCDC into Iceberg takes the plain commit path.

The Hive profile found four product defects that the memory catalog could not show:

1. `file:/path` was read as relative to the server's working directory, for every catalog. It is now absolute.
2. Every Hive table read back as external, so a materialized view could not refresh. Owned tables now carry `external.table.purge`, and DROP deletes their data, as does DROP DATABASE.
3. Listing tables failed on Hive 4.0.1, which removed `get_table_objects_by_name`. The newer request form is tried first, with the old call as fallback.
4. Hive names map entries `keys` and `values`. Nested write validation now compares map entries by position, as Spark does.

Hive cannot REPLACE, so a fenced reset overwrites by path and a plain reset drops and creates. When Unity refuses PURGE, the drop falls back to a non-PURGE drop.

5 Verification methodology

5.1 Principles

The gates were built to stop two failure modes: counting a pass that was not real, and missing a regression outside the pipelines module. The rules (§15):

- **Spark first.** Spark 4.2.1 is measured before Sail behavior is accepted. A test that fails on Spark is rerun alone, in its own session with an empty warehouse, before it counts as a difference. In the first such run, 32 of 40 Spark failures were leftovers from earlier tests sharing the `default` schema.
- **Controls.** A control run with the fix reverted must fail the new tests.
- **Honest records.** Failed runs are kept and never relabelled. No upstream assertion is removed, no xfail is added, and no table is swapped for a materialized view (or streaming for batch) to get a pass. No debug RPC or Spark-internal clone is added to satisfy fixtures.

- **Frozen sources.** Changing a frozen test source requires a named alignment record in the harness `preparation.json`.
- **Immutable gates.** Each verdict names one detached commit with its own target directory. The toolchain is Rust 1.98.1 with `--locked`, nightly fmt per crate (never `cargo fmt --all`, which rewrites `vendor/`), strict Clippy, and a Python 3.12 venv with PySpark 4.2.1.
- **Landing.** Only through `gate-run.sh` then `land.sh`, or `gate-merge.sh` then a fast-forward, and only after the frozen harness prints PASS.

5.2 Frozen ports

Spark's internal Scala suites instantiate Catalyst and Spark internals; they cannot run against a Rust server. The user excluded them from the goal. Instead, 300 selected public tests and 67 upstream Python tests are frozen, with hashes, and run on every gate. They must print PASS.

An audit of the non-AutoCDC Scala suites maps each case to a Sail test. Final state: 384 ported, 37 partial, 14 internal, 0 missing. The 37 partial entries are wire-invisible: the Scala assertion checks internal state the Connect protocol does not expose (event fields and capacity that never reach a client, states a client cannot request), plus AutoCDC cases of PythonPipelineSuite and the `memory` writeStream case. The user accepted them as finished scope.

5.3 Gate steps

The batch lane's gate, which lands every commit on the root, runs:

- native tests on 15 crates;
- the frozen 300 + 67;
- the pipelines and ORC Python suites;
- the CLI test, with `catalog.shared_memory` on;
- the Hive metastore profile and the second-catalog profile;
- the Delta suite, compared by failure name with the baseline.

The CDC lane's gate adds, for every offer:

- fmt and strict Clippy;
- the Spark Connect, Delta, control-store and sail-session libraries;
- the whole Spark Python tree;
- local-cluster, Iceberg, Hive and Unity runs;
- the Spark oracle, `crash.py` seeds and `iceberg_on_spark` (the AutoCDC files on Spark with Iceberg).

Gates launch with SIGINT unblocked. A blocked SIGINT is inherited by every Sail server, and the Hive script then hangs in teardown.

5.4 Whole-tree comparison against upstream

For a while the whole Spark Python tree was compared only with earlier roots of this branch. "No new failures" then meant none since the last root. Run against the upstream merge base `0ba425c03`, 75 tests passed upstream and failed on the branch. The causes were general changes: the ANSI string cast lowering broke correlated subquery decorrelation (56 tests) and VALUES NaN detection; a CASE coercion used the non-ANSI policy; unresolved-column reporting failed on internal columns (8 tests); the partition-column check refused nested sources. All were fixed.

A second correction followed. TPC-H, TPC-DS and two utility tests had never collected because duckdb and jedi were missing, and other tests failed for harness reasons. `run-full.py` pins the optional test dependencies as in `pyproject.toml`. Under it, upstream `0ba425c03` has 0 failures and 12 JVM fixture errors. The CDC lane's gate on the milestone commit `7d6bc1917` ran the whole tree: 12134 passed, 99 skipped, 795 xfailed, 0 errors.

5.5 Spark oracle runs

Batch. Batch test files are identical on `df2457e50` and on the root. Every non-AutoCDC pipelines file, 61 files, was run on Spark 4.2.1 (delta-spark 4.4.1, `local[2]`), each in its own Spark session. That run had 40 failures. Rerun alone with an empty warehouse, 36 passed. The 4 that fail on Spark alone are all documented Sail differences:

- `test_rejected_flow_does_not_execute_insert`: Sail rejects before the INSERT child runs.
- `test_a_failed_sql_file_registers_no_partial_definitions`: file-atomic registration.
- `test_quoted_identifier_with_dot` and `test_escaped_hyphenated_dataset_dependencies`: Spark's session catalog refuses these names.

Before the `2e73a0b87` fixes the same run had 8 real differences; bare `USE`, declared view columns and `DATA_SOURCE_NOT_FOUND` now pass on Spark too.

AutoCDC. Plain Spark lacks a catalog whose tables support row-level MERGE, so the AutoCDC oracle uses a row-level test catalog. With Iceberg, Spark needs the Iceberg 1.13.0-SNAPSHOT nightly for Spark 4.2; the 1.12.0 runtime, built for Spark 4.1, does not load. On this root, `test_autocdc_semantics.py`: 41 passed; `test_autocdc_spark_suites.py`: 43 passed and 4 skipped (Sail-only cases). Spark with Iceberg refuses 3 nested-type AutoCDC cases that Sail completes.

5.6 Crash seeds and local-cluster

`crash.py` kills the server with SIGKILL at seeded points of the durable AutoCDC run and checks that the next run recovers exactly. On the CDC gate every recovery was exact. Separately, the pipelines suite runs in local-cluster mode: 95 passed on the CDC gate. The racy release tests were repeated 12 rounds in local-cluster mode while fixing the driver hang: the fixed binary was clean all 12 rounds, with the 30-second bound firing once; the prior binary failed one round.

5.7 Results on the root

Gate	Commit	Result
Native tests	`fb3a6ab03`	15 crates
Frozen 300 + 67	`fb3a6ab03`	PASS
Pipelines and ORC	`fb3a6ab03`	1262 passed, 23 skipped, 4 strict xfail
CLI, shared memory on	`fb3a6ab03`	passed
Hive metastore	`fb3a6ab03`	69 passed, 2 xfailed
Second catalog	`fb3a6ab03`	13 passed
Delta suite	`fb3a6ab03`	same failures by name as baseline
Shutdown	`fb3a6ab03`	no session cleanup failures
Whole Spark Python tree	`7d6bc1917`	12134 passed, 0 errors
Local-cluster	`7d6bc1917`	96 passed
Iceberg / Hive / Unity	`7d6bc1917`	87 / 89 / 230 passed
Crash seeds	`7d6bc1917`	every recovery exact

6 Differences from Spark

6.1 Strict expected failures

These are pinned as strict expected failures and accepted as finished scope (user decision 6).

Difference	Reason
Interrupting a run stops its flows. Spark lets them finish.	Otherwise nothing could stop such a run, and session shutdown needs the flows stopped.
Releasing a session during a run stops its flows.	Same reason.
Added columns follow the target table's order.	Spark's order comes from a Scala Map and is unstable past four fields.
`writeStream` to `memory` outside a pipeline is not supported.	Not in scope; inside a pipeline the memory sink works.

A union of streaming and batch inputs was once a fifth. Since `609a6c251` Sail refuses it, as Spark does.

6.2 Other deliberate differences

Difference	Reason
A SQL file registers all-or-nothing.	Spark keeps definitions before a failing statement, but the client aborts the run anyway. All-or-nothing leaves no partial graph.
A rejected flow is rejected before its INSERT runs.	Spark writes the INSERT child first. Sail makes no effect for a statement it refuses.
Dotted and hyphenated table names are accepted.	Sail's engine-wide identifier policy; not pipeline-specific.
`DATA_SOURCE_NOT_FOUND` arrives unwrapped.	Spark wraps it in `TableMaterializationException` inside `UnknownException`. The text matches; the class difference is disclosed.
Failure events are sorted.	Scala's Map order is unspecified.
A view that repeats a column is refused during analysis.	Spark accepts `SELECT *` over it and refuses a reference.
ORC instants in ORC-763's pre-epoch window fail the write.	Spark writes them and reads back a wrong value.
AutoCDC ties keep the last row.	Spark leaves ties undefined.
3 Iceberg nested-type AutoCDC cases complete.	Spark with Iceberg 1.13 refuses them; Sail keeps completing them (user decision 12).
Delta AutoCDC targets are accepted.	Spark refuses with `AUTOCDC_TARGET_DOES_NOT_SUPPORT_MERGE`. Recorded, but not yet confirmed as policy.

7 Scope decisions, limits and open work

7.1 Scope the user set

- Spark's internal Scala suites are excluded. The frozen 300 + 67 ports stand in for them.
- Docker- and S3-backed Hive tests are waived because Docker Desktop was down. The Derby Hive profile runs instead. Remote warehouses are tested against `moto`, not real S3.
- Unity Catalog is built from source and run locally.
- The 4 strict expected failures and the 37 wire-invisible partial audit entries are accepted.
- Only process death is a qualified crash. Power loss and restore are not.
- Delivery is a pushed milestone branch and tag, authorized by the user for this milestone. There is no release, and unqualified effects are never activated on the shared branch.

7.2 Architectural limits

- **Flows run one at a time.** Execution walks the topological order sequentially. Independent flows do not run concurrently, although each flow's plan runs in parallel through `JobService`.
- **AvailableNow only.** There is no continuous streaming. The append path admits a stateless subset plus tumbling-window aggregates.

- **Plain file append is at-least-once.** A crash replays exactly the recorded batch, which can duplicate rows in a file table with no idempotent commit.
- **AutoCDC state is local.** The Turso authority, archives and file lists live on one machine, even for a remote warehouse. V1 is one durable service with no HA claim.
- **SCD1 only.** SCD2 is rejected, as in Spark.
- **Cluster release with checkpoints fails.** The driver cannot confirm remote writers stopped, so it keeps the files and the server exits 1.
- **ORC.** No hybrid-calendar rebase; memory reservations are estimates; not run on real object stores.
- **Hive.** The fallback to older metastores and S3-backed Hive are unverified.

7.3 Open work

Item	State
Late rows in the append path	Sail drops late rows by event time. Spark drops a row only when its window has ended (`window.end <= watermark`, with watermark = floorDiv(max event µs, 1000) − delay ms, starting at 0, never moving back). The next unit of work.
Delta AutoCDC targets	Whether accepting them is policy.
Worker stop	A worker can wait up to 5 seconds, because a canceled job's stage-0 Flight stream stays open after its tasks are aborted.
CDC durability	Production durability, fencing and recovery beyond the qualified process-death cases: the activation gate, a real backend qualification, the recovery trace.
CDC representation	Logical canonical encoding or sealed bytes; the tie-break; durable staging.
Session work	Fallible actor and gateway results; routing non-checkpoint writers through the effect owner; cluster worker effects.
Append sources	Delta and Iceberg append sources beyond the current reader, rate limits, exactly-once for plain file tables.
Dynamic references	Frozen SQL inputs; configured dynamic and With-Relations binding.
Performance	Replay fast path, constant folding, lazy NaNvl.
Protocol	Deferred query-function signals and `ExecuteOutputFlows`, which Spark 4.2.1 does not dispatch either.

8 The demo

The branch carries a demo at `examples/declarative-pipelines/` that proves the features above on one machine, after a checkout and a build. Its README is the full guide; this chapter says what it shows.

8.1 Running it

```
cargo build --release -p sail-cli
python3 -m venv .venv && .venv/bin/pip install 'pyspark[connect]==4.2.1' pyyaml
PYTHON=.venv/bin/python examples/declarative-pipelines/run.sh
```

To include Apache Iggy, build `iggy-server` at the revision the branch pins (`153fdf466aea94b7bcb90681ee7274f9f76ec00d`) and name it:

```
IGGY_SERVER_BINARY=/path/to/iggy-server PYTHON=.venv/bin/python examples/declarative-pipelines/run.sh
```

Without a broker the Iggy part is reported as skipped, never as passed.

8.2 What it does

`run.sh` starts a Sail server whose sessions share one in-memory catalog, as Spark's do. The driver, `demo.py`, then runs pipeline projects through the unchanged Spark 4.2.1 `spark-pipelines` CLI. After each step it checks the results through its own Spark Connect session and prints PASS or FAIL. The exit status is 0 only when every check passed.

Step	What it proves
Dry run	The graph is analyzed and nothing is written.
First run	SQL and Python datasets; a streaming table over a table; a materialized view; a temporary view that is never published; a persisted view; comments; an ORC materialized view; JSON-file and Delta streaming sources; two append flows into one table; AutoCDC into Delta.
Second run	Streaming tables read only what is new. AutoCDC applies a delete and an insert, and ignores an out-of-order update.
Third run	With no new data, nothing is read twice.
Selective refresh	`--refresh daily_revenue` runs only that dataset.
Full refresh	`--full-refresh-all` rebuilds streaming tables from their sources and replays the change feed to the same AutoCDC state.
Broken graph	An unresolved column fails the run with `UNRESOLVED_COLUMN` before any dataset, valid or not, is written.
Iggy	A streaming table and an AutoCDC target read one topic. Later runs read only new messages.
Shutdown	The server stops cleanly on SIGINT.

On the milestone commit the demo reports 48 passed, 0 failed, 0 skipped, with the Iggy broker.

8.3 What it found

The first run of the demo failed every AutoCDC step. Sail's default warehouse is the relative path `spark-warehouse`. The AutoCDC commit path built its table URL without resolving a relative location, and the tests had never noticed because they always configure an absolute warehouse. The fix resolves the location with `sail_data_source::resolve_table_url`, the resolver Delta's table source uses, so both name the same directory. It is part of this milestone.

For the test suites, `scripts/spark-pipelines-tests/` holds the runners, including `run-hive-metastore.sh` and `run-second-catalog.sh`. Tests that need a broker read `SAIL_IGGY_SERVER_BINARY`. Unity tests read `PYSAIL_UNITY_CATALOG_URL`. An external test server needs `SAIL_EXECUTION__DEFAULT_PARALLELISM=4` and a 16 MiB stack.